Python Notes (Lecture 6)

Functions

1. Functions in Python

A function is a **block of statements that performs a specific task**. We write it once and can use it many times.

```
def func_name(param1, param2):    # function definition
    # some work
    return val

func_name(arg1, arg2)              # function call
```

- `def` keyword starts a function definition.
- **Parameters** are the names in the definition; **arguments** are the actual values passed in the call.
- `return` sends a value back to where the function was called.
- A function runs only when it is **called**.

```
def calc_sum(a, b):
    s = a + b
    return s

print(calc_sum(2, 3))
print(calc_sum(10, 20))
```

Output:

```
5
30
```

A function without `return` gives back `None`:

```
def greet(name):
    print("Hello,", name)

result = greet("Aman")
print(result)
```

Output:

```
Hello, Aman
None
```

---

2. Types of Functions

Built-in Functions	User-defined Functions
Already provided by Python	Written by the programmer
`print()`, `len()`, `type()`, `range()`	Created with `def`, like `calc_sum()` above

Tip: don't give your function the same name as a built-in one (for example `sum` or `len`), or the built-in stops working in that program.

---

3. Default Parameters

A default value can be given to a parameter. It is used when **no argument is passed** for it.

```
def calc_product(a=1, b=2):
    return a * b

print(calc_product())        # uses both defaults
print(calc_product(5))       # a = 5, b uses default
print(calc_product(5, 6))
```

Output:

```
2
10
30
```

Parameters with defaults must come **after** those without: `def f(a, b=2)` is valid, `def f(a=1, b)` is not.

Let's Practice

Q1. Write a function to print the length of a list (list is the parameter).

```
def print_len(items):
    print(len(items))

cities = ["Delhi", "Mumbai", "Pune"]
print_len(cities)
```

Output:

```
3
```

Q2. Write a function to print the elements of a list in a single line (list is the parameter).

```
def print_list(items):
    for item in items:
        print(item, end=" ")
    print()

print_list(["Delhi", "Mumbai", "Pune"])
```

Output:

```
Delhi Mumbai Pune
```

`end=" "` makes `print()` end with a space instead of a new line.

Q3. Write a function to find the factorial of n (n is the parameter).

```
def factorial(n):
    fact = 1
    for i in range(1, n + 1):
        fact *= i
    return fact

print(factorial(5))
```

Output:

```
120
```

Q4. Write a function to convert USD to INR.

```
def usd_to_inr(usd, rate=83):
    return usd * rate

print(usd_to_inr(10))           # uses default rate
print(usd_to_inr(10, 88))       # pass the current rate
```

Output:

```
830
880
```

The exchange rate keeps changing, so it is a parameter with a default value. Pass the current rate when you use it.

Recursion

1. What is Recursion?

Recursion is when a **function calls itself** repeatedly. Every recursive function needs a **base case**: a condition where it stops calling itself. Without it, the function would run forever (and Python stops it with a `RecursionError`).

Print n to 1 backwards:

```
def show(n):
    if n == 0:          # base case
        return
    print(n)
    show(n - 1)         # function calls itself

show(5)
```

Output:

```
5
4
3
2
1
```

---

2. Factorial using Recursion

n! = n × (n−1)!, and 0! = 1! = 1 (base case).

```
def fact(n):
    if n == 0 or n == 1:      # base case
        return 1
    else:
        return n * fact(n - 1)

print(fact(5))
```

Output:

```
120
```

How `fact(4)` works:

```
fact(4) = 4 * fact(3)
        = 4 * 3 * fact(2)
        = 4 * 3 * 2 * fact(1)
        = 4 * 3 * 2 * 1
        = 24
```

Let's Practice : Recursion

Q1. Write a recursive function to calculate the sum of the first n natural numbers.

```
def calc_sum(n):
    if n == 0:                    # base case
        return 0
    return n + calc_sum(n - 1)

print(calc_sum(5))
```

Output:

```
15
```

calc_sum(5) = 5 + 4 + 3 + 2 + 1 + 0 = 15.

Q2. Write a recursive function to print all elements in a list.

Hint: use the list and an index as parameters.

```
def print_list(items, idx=0):
    if idx == len(items):         # base case: reached the end
        return
    print(items[idx])
    print_list(items, idx + 1)

print_list(["mango", "apple", "banana"])
```

Output:

```
mango
apple
banana
```